LIBRARY MANAGEMENT SYSTEM USING DATA STRUCTURES

PRESENTING BY

NAME : ZAHEDUL ISLAM
ID : 2024-3-60-561

NAME : MD RAHATUL ISLAM RATUL
ID : 2024-3-60-677

NAME : AHMAD MASUD
ID : 2024-3-60-627

SYSTEM OVERVIEW

- Library Management System for efficient operations
- Doubly Linked List for storing and browsing books
- Queue for fair reservation handling (FIFO)
- Hash Map for fast book lookup (O(1))
- Stack for tracking history and undo actions
- Combines all structures for a simple and efficient system

CORE DATA STRUCTURES

- Doubly Linked List stores the full library catalog
- Allows easy forward and backward traversal of books
- Each node contains book details
- Uses prev and next pointers for smooth navigation
- Supports efficient browsing and management of the catalog

- Queue manages book reservation requests
- Used when a book is already borrowed
- Follows First-Come-First-Served (FIFO) rule
- Each book has its own reservation queue
- Ensures fair and orderly access for waiting users

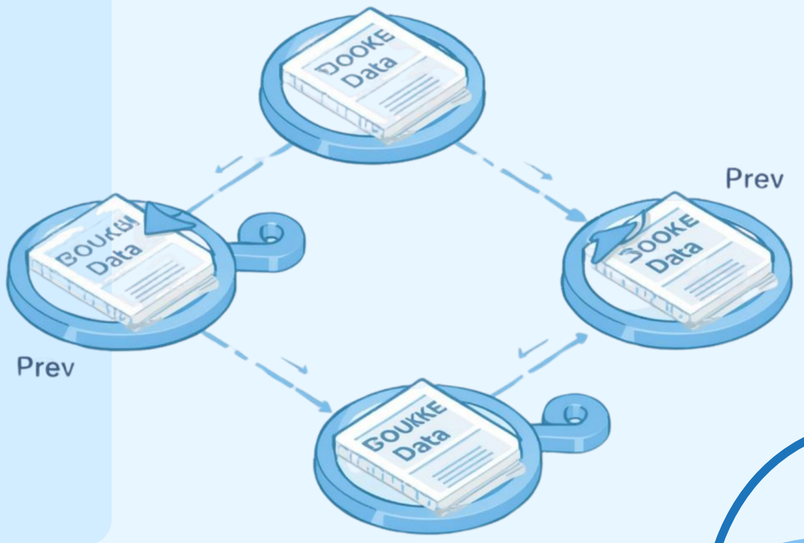
CORE DATA STRUCTURES

- Hash Map stores mapping from Book ID to BookNode pointer
- Provides instant O(1) lookup for books
- Enables fast retrieval of book status and location
- Implemented using bookMap data structure
- Ensures high efficiency in search and access operations

- Stack stores history of borrow and return actions
- Uses LIFO (Last-In-First-Out) order
- Supports push and pop operations
- Stores action type, bookId, and userId
- Enables undo functionality for recent transactions

DOUBLY LINKED LIST FOR CATALOG

Purpose: Store and browse the complete library catalog with efficient bidirectional traversal.
Benefits: Navigate forward and backward through books seamlessly, with O(1) insertion at head or tail.
BookNode Structure:
• id, title, author
• isAvailable, reservationQueue
• prev and next pointers
Maintains head and tail pointers for quick access to catalog boundaries.



DOUBLY LINKED LIST PSEUDO CODE

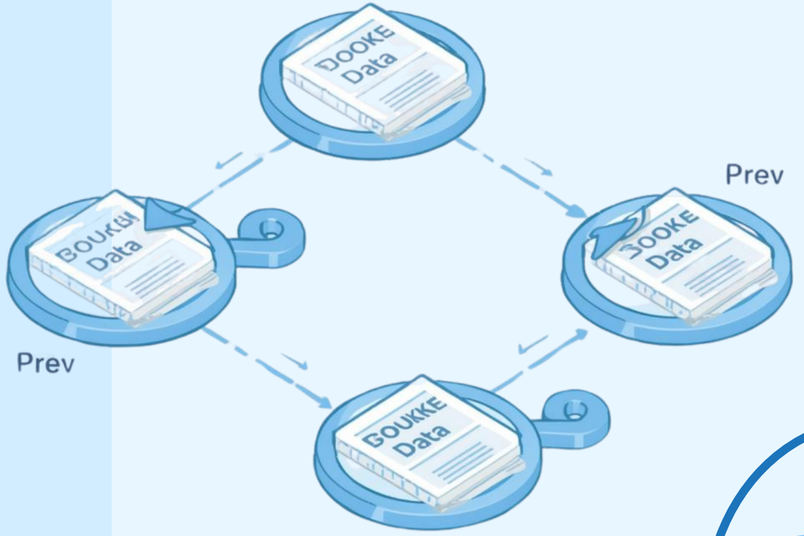
```
STRUCT BookNode
    id, title, author
    isAvailable
    reservationQueue
    prev, next
END STRUCT

INITIALIZE head = NULL
INITIALIZE tail = NULL

FUNCTION addBook(id, title, author)
    CREATE newNode

    IF head == NULL
        head = newNode
        tail = newNode
    ELSE
        tail.next = newNode
        newNode.prev = tail
        tail = newNode
    ENDIF

END FUNCTION
```



QUEUE FOR RESERVATION MANAGEMENT

- Queue manages reservation requests for borrowed books
- Each book has its own reservationQueue
- Follows First-Come-First-Served (FIFO) order
- New users are added to the back of the queue
- On return, the first user is dequeued and assigned the book
- Ensures fair and equal access for all users

QUEUE PSEUDO CODE

```
FUNCTION borrowBook(book, userId)
    IF book.isAvailable == TRUE
        book.isAvailable = FALSE
        PRINT "Book borrowed"
    ELSE
        ENQUEUE userId into book.reservationQueue
        PRINT "Added to queue"
    ENDIF
END FUNCTION

FUNCTION returnBook(book)
    IF queue is NOT empty
        nextUser = FRONT of queue
        DEQUEUE
        PRINT "Assigned to next user"
    ELSE
        book.isAvailable = TRUE
        PRINT "Book available"
    ENDIF
END FUNCTION
```

HASH MAP FOR FAST LOOKUP

Purpose: Store mapping from book ID to BookNode pointer for instant retrieval.

Benefit: O(1) constant time lookup for book status and retrieval operations.

Implementation: The bookMap data structure stores key-value pairs where keys are unique book IDs and values are pointers to BookNode objects in the doubly linked list.

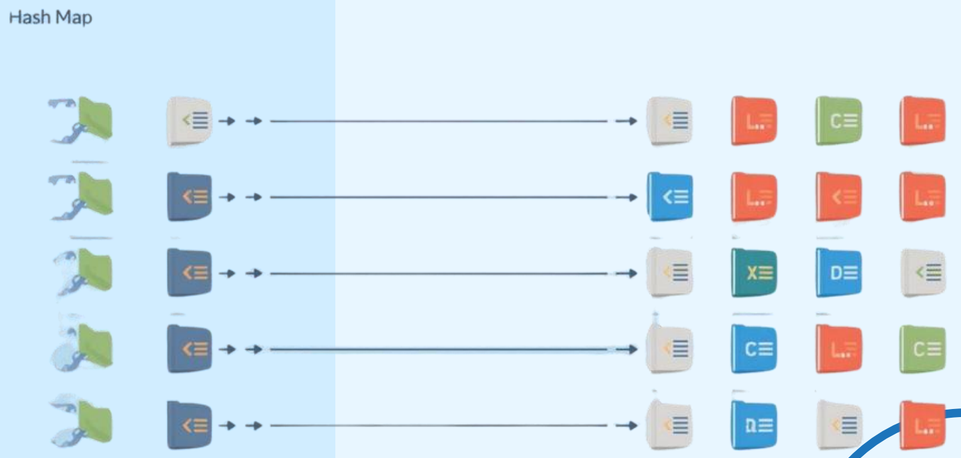
This enables fast search, borrow, and return operations without traversing the entire catalog.

HASH MAP PSEUDO CODE

```
INITIALIZE bookMap

FUNCTION insertToMap(id, node)
    bookMap[id] = node
END FUNCTION

FUNCTION searchBook(id)
   IF id exists in bookMap
       RETURN bookMap[id]
   ELSE
       PRINT "Book not found"
    ENDIF
END FUNCTION
```



STACK FOR HISTORY AND UNDO

- Stack stores history of all borrow and return actions
- Each entry includes action type, bookId, and userId
- New actions are pushed onto the stack
- undoAction() pops the most recent action
- Reverses the operation to restore previous state
- Helps correct mistakes and maintain data integrity

STACK PSEUDO CODE

```
INITIALIZE historyStack

FUNCTION pushAction(type, bookId, userId)
   PUSH (type, bookId, userId) into historyStack
END FUNCTION

FUNCTION undoAction()
   IF stack is empty
      PRINT "No action to undo"
      RETURN
   ENDIF

   action = POP from stack

   IF action.type == "BORROW"
      mark book as available
   ELSE IF action.type == "RETURN"
      mark book as borrowed
   ELSE IF action.type == "ADD"
      remove or hide book
   ENDIF
END FUNCTION
```

SYSTEM MODULES

Add Book to Catalog: Insert new books into the library system using the addBook function. Creates a new BookNode, appends it to the doubly linked list, updates head/tail pointers, and registers the book ID in the hash map for instant retrieval.

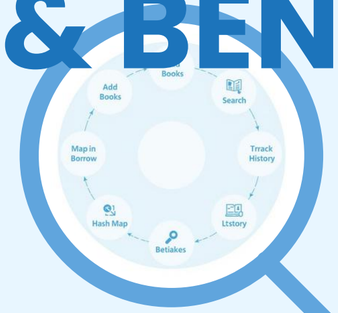
Search Book: Quickly locate any book using **O(1)** hash map lookup by book ID. The searchBook function retrieves the BookNode pointer directly from bookMap, returning complete book details including availability status and reservation queue.

OBJECTIVES

Borrow Book: Check book availability via hash map lookup. If available, mark as borrowed and push action to history stack. If unavailable, enqueue user in the book's reservationQueue for fair waiting.

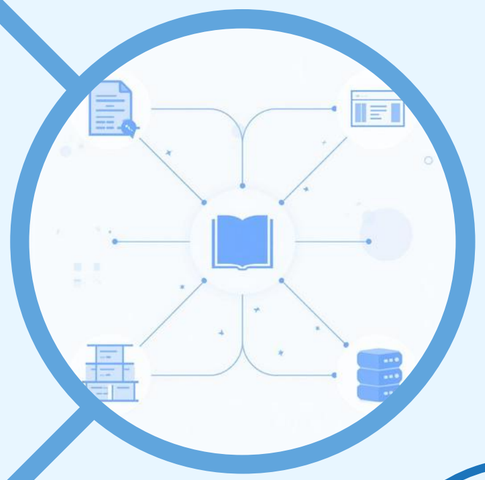
Return Book: Process returns by checking reservationQueue. If users are waiting, automatically assign book to next user in queue. Otherwise, mark book as available. Push return action to history stack for undo capability.

SYSTEM WORKFLOW & BENEFITS



Workflow Steps:
Add books to catalog → Map in hash map → Search via hash map → Borrow with availability check and queue → Return with queue assignment → Track actions in history stack for undo.

Key Benefits: Efficient catalog browsing with doubly linked list, fair first-come-first-served reservation management, **O(1)** fast book lookup via hash map, and reversible operations through history stack.



THANK YOU!

Questions? Library Management System Using Data Structures